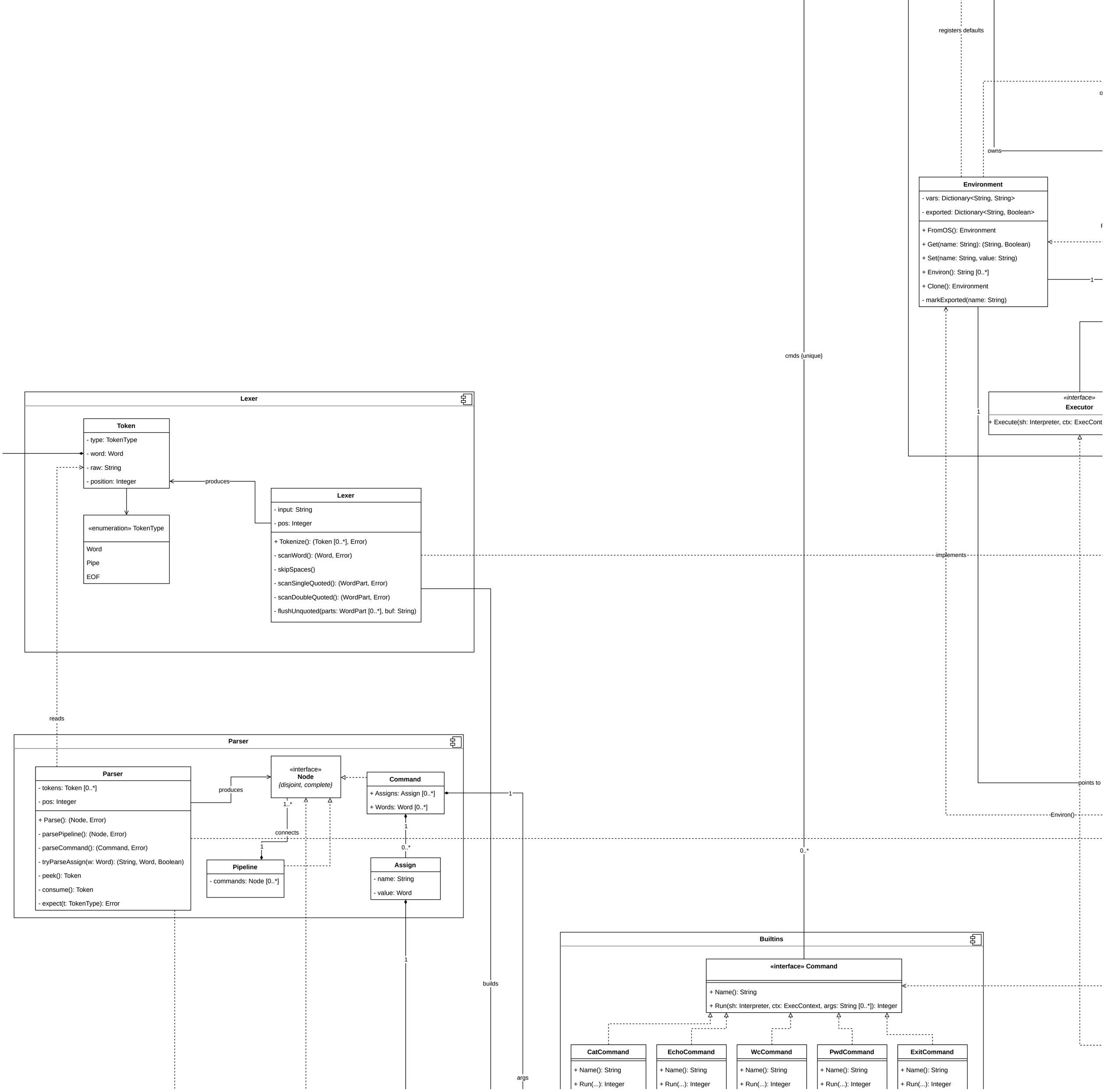
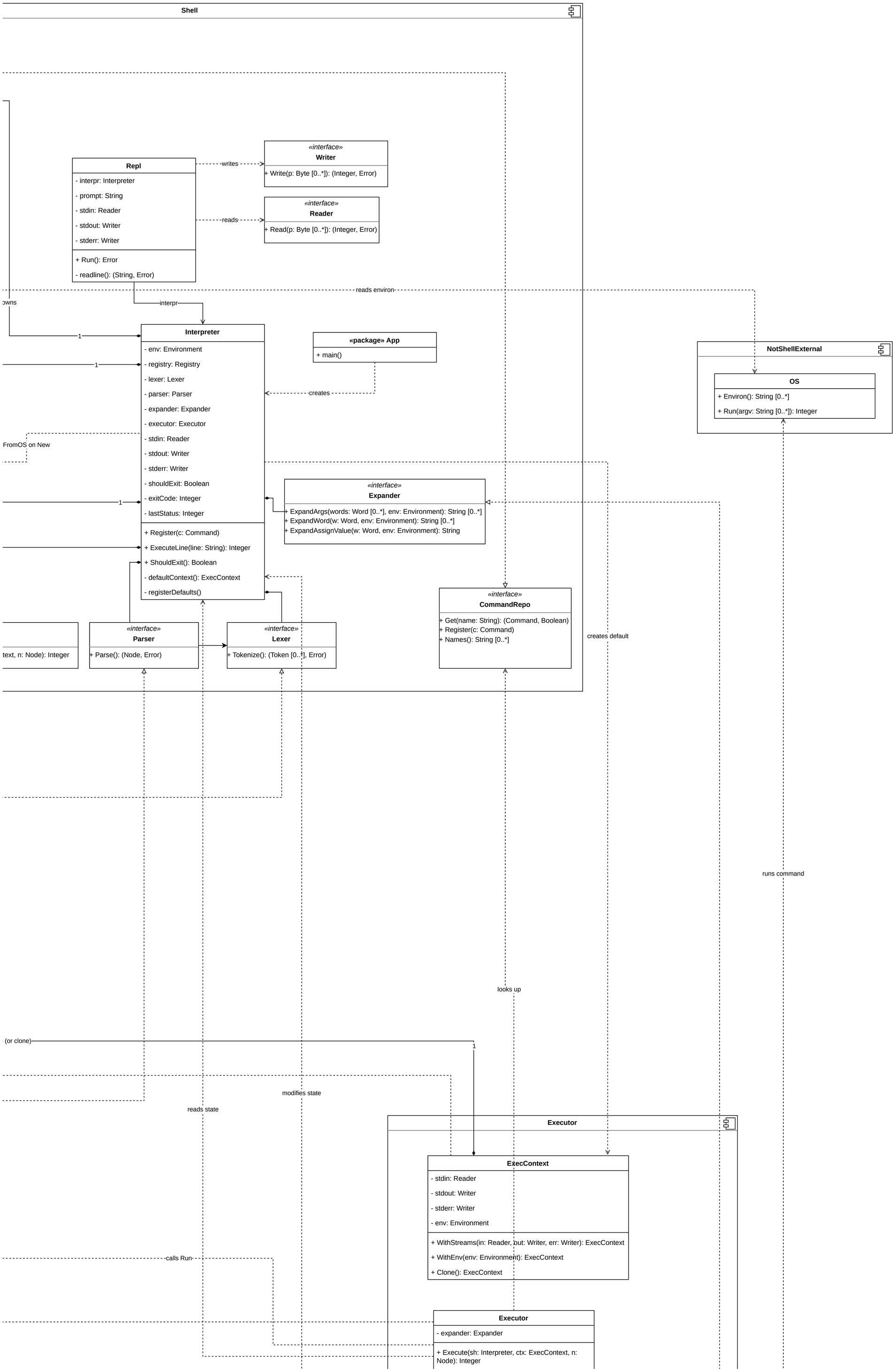
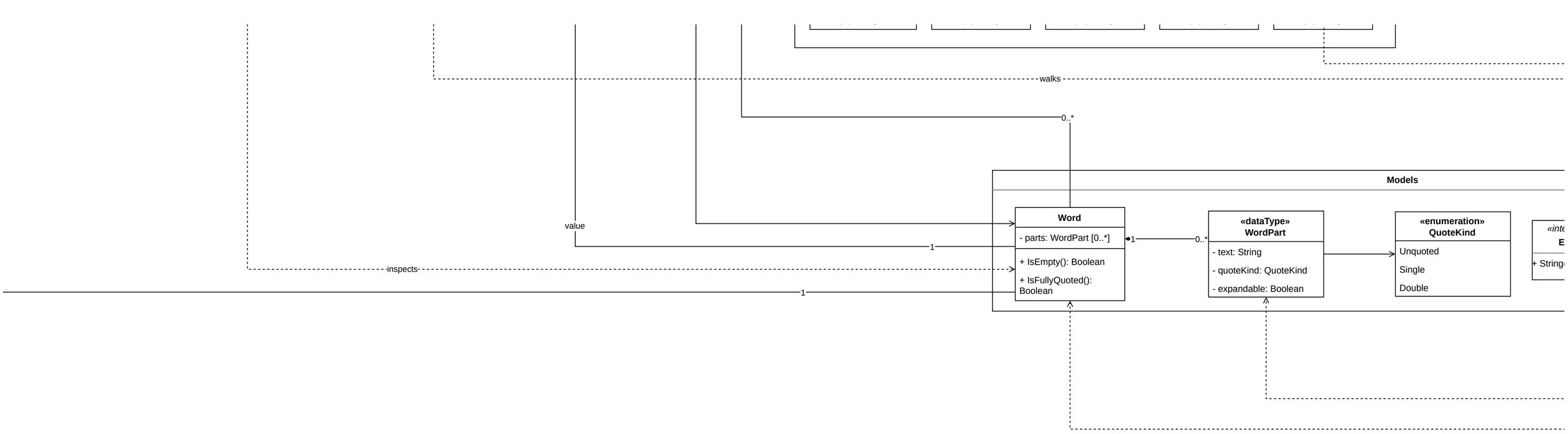
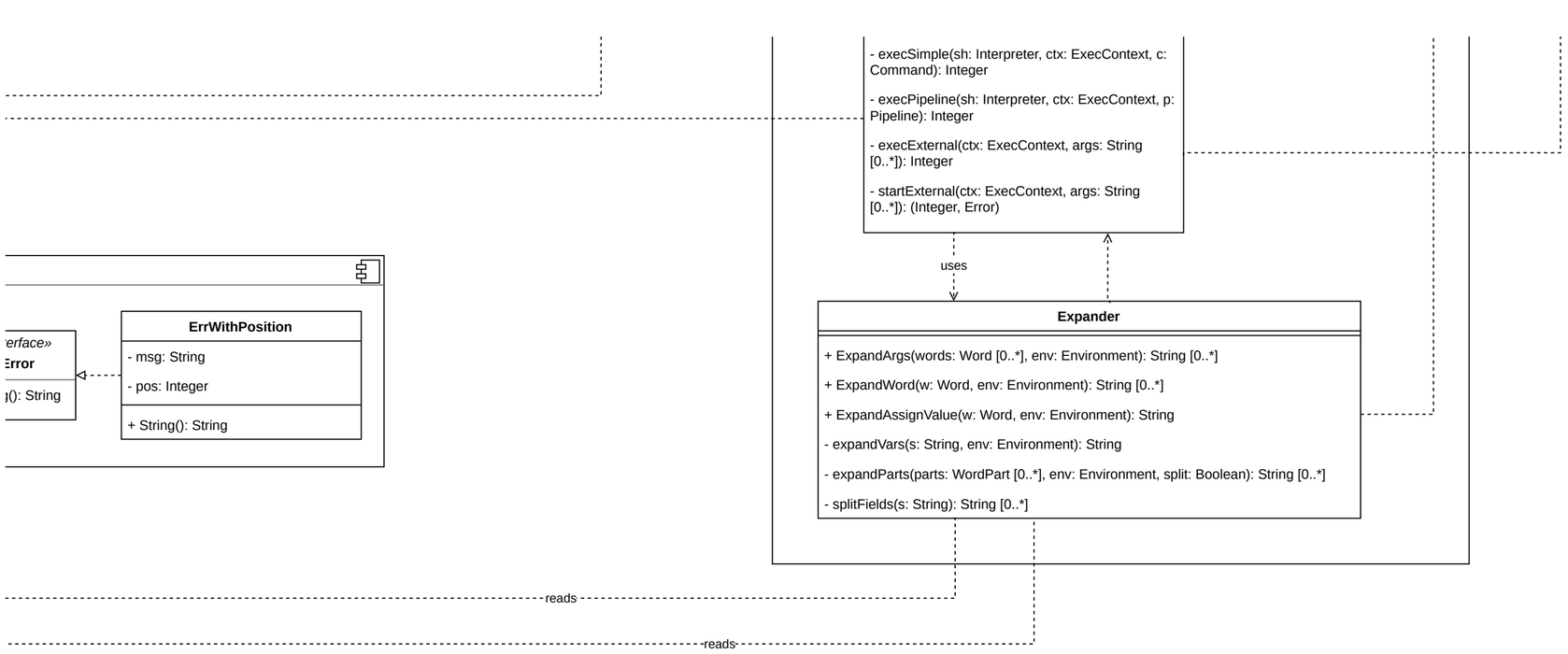
1


Registry
- cmds: Dictionary<String, Command>
+ Get(name: String): (Command, Boolean)
+ Register(c: Command)
+ Names(): String [0..*]
1
1
1
registers defaults
1
o
owns
Environment
- vars: Dictionary<String, String>
- exported: Dictionary<String, Boolean>
+ FromOS(): Environment
+ Get(name: String): (String, Boolean)
+ Set(name: String, value: String)
+ Environ(): String [0..*]
+ Clone(): Environment
- markExported(name: String)
1
cmds {unique}
«interface»
Executor
+ Execute(sh: Interpreter, ctx: ExecCont
1
Lexer
Token
- type: TokenType
- word: Word
- raw: String
- position: Integer
produces
«enumeration» TokenType
Word
Pipe
EOF
Lexer
- input: String
- pos: Integer
+ Tokenize(): (Token [0..*], Error)
- scanWord(): (Word, Error)
- skipSpaces()
- scanSingleQuoted(): (WordPart, Error)
- scanDoubleQuoted(): (WordPart, Error)
- flushUnquoted(parts: WordPart [0..*], buf: String)
implements
reads
Parser
Parser
- tokens: Token [0..*]
- pos: Integer
+ Parse(): (Node, Error)
- parsePipeline(): (Node, Error)
- parseCommand(): (Command, Error)
- tryParseAssign(w: Word): (String, Word, Boolean)
- peek(): Token
- consume(): Token
- expect(t: TokenType): Error
produces
«interface»
Node
{disjoint, complete}
Command
+ Assigns: Assign [0..*]
+ Words: Word [0..*]
1..*
connects
1
1
0..*
Pipeline
- commands: Node [0..*]
Assign
- name: String
- value: Word
1
points to
Environ()
0..*
builds
args
Builtins
«interface» Command
+ Name(): String
+ Run(sh: Interpreter, ctx: ExecContext, args: String [0..*]): Integer
CatCommand
+ Name(): String
+ Run(...): Integer
EchoCommand
+ Name(): String
+ Run(...): Integer
WcCommand
+ Name(): String
+ Run(...): Integer
PwdCommand
+ Name(): String
+ Run(...): Integer
ExitCommand
+ Name(): String
+ Run(...): Integer


Shell
Repl
- interpr: Interpreter
- prompt: String
- stdin: Reader
- stdout: Writer
- stderr: Writer
+ Run(): Error
- readline(): (String, Error)
writes
«interface»
Writer
+ Write(p: Byte [0..*]): (Integer, Error)
reads
«interface»
Reader
+ Read(p: Byte [0..*]): (Integer, Error)
reads environ
interpr
owns
1
1
1
Interpreter
- env: Environment
- registry: Registry
- lexer: Lexer
- parser: Parser
- expander: Expander
- executor: Executor
- stdin: Reader
- stdout: Writer
- stderr: Writer
- shouldExit: Boolean
- exitCode: Integer
- lastStatus: Integer
+ Register(c: Command)
+ ExecuteLine(line: String): Integer
+ ShouldExit(): Boolean
- defaultContext(): ExecContext
- registerDefaults()
«package» App
+ main()
creates
FromOS on New
«interface»
Expander
+ ExpandArgs(words: Word [0..*], env: Environment): String [0..*]
+ ExpandWord(w: Word, env: Environment): String [0..*]
+ ExpandAssignValue(w: Word, env: Environment): String
«interface»
Parser
+ Parse(): (Node, Error)
«interface»
Lexer
+ Tokenize(): (Token [0..*], Error)
«interface»
CommandRepo
+ Get(name: String): (Command, Boolean)
+ Register(c: Command)
+ Names(): String [0..*]
creates default
NotShellExternal
OS
+ Environ(): String [0..*]
+ Run(argv: String [0..*]): Integer
runs command
looks up
(or clone)
1
modifies state
reads state
Executor
ExecContext
- stdin: Reader
- stdout: Writer
- stderr: Writer
- env: Environment
+ WithStreams(in: Reader, out: Writer, err: Writer): ExecContext
+ WithEnv(env: Environment): ExecContext
+ Clone(): ExecContext
calls Run
Executor
- expander: Expander
+ Execute(sh: Interpreter, ctx: ExecContext, n: Node): Integer


walks
0..*
Models
Word
- parts: WordPart [0..*]
+ IsEmpty(): Boolean
+ IsFullyQuoted(): Boolean
«dataType»
WordPart
- text: String
- quoteKind: QuoteKind
- expandable: Boolean
«enumeration»
QuoteKind
Unquoted
Single
Double
value
1
1
0..*
inspects
1


- execSimple(sh: Interpreter, ctx: ExecContext, c: Command): Integer
- execPipeline(sh: Interpreter, ctx: ExecContext, p: Pipeline): Integer
- execExternal(ctx: ExecContext, args: String [0..*]): Integer
- startExternal(ctx: ExecContext, args: String [0..*]): (Integer, Error)
uses
Expander
+ ExpandArgs(words: Word [0..*], env: Environment): String [0..*]
+ ExpandWord(w: Word, env: Environment): String [0..*]
+ ExpandAssignValue(w: Word, env: Environment): String
- expandVars(s: String, env: Environment): String
- expandParts(parts: WordPart [0..*], env: Environment, split: Boolean): String [0..*]
- splitFields(s: String): String [0..*]
ErrWithPosition
- msg: String
- pos: Integer
+ String(): String
reads
reads